Lecture 12: Graph Coloring for Real Life Problems

From conflict graphs to a feasible, balanced schedule

Graph Coloring → **Multiple Real Constraints** → **Timetable Optimization**

Case Study: Clash-free scheduling under limited rooms, time slots and shared faculty

Simple Coloring Model

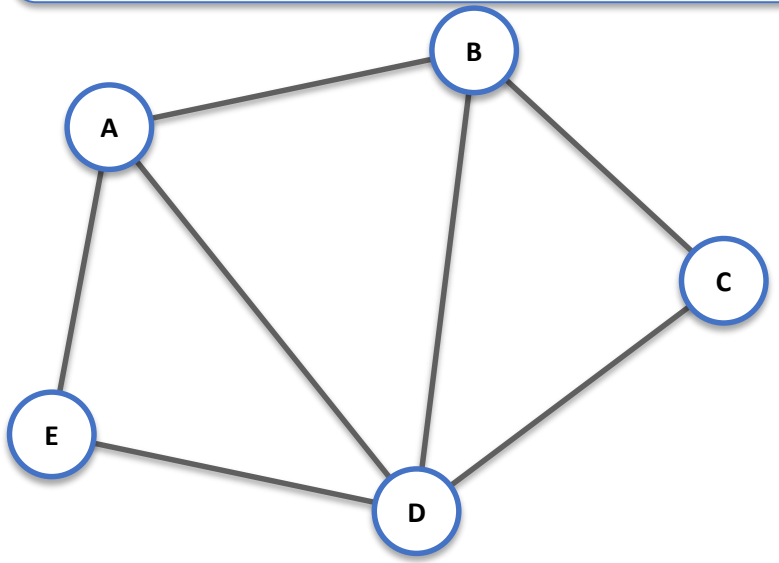
Vertex = a class / course section

Edge = two classes cannot be scheduled at the same time

Color = a time slot

Adjacent vertices must receive different colors.

Conflict Graph

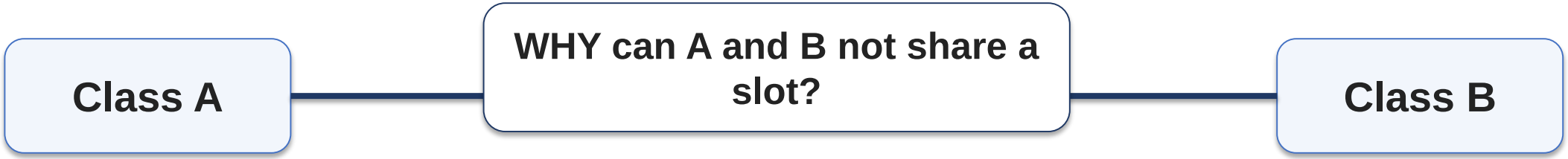


Goal: assign available time slots without conflicts.

But the Real University Timetable Is Not Just This

Student clashes	Shared students / batches / electives
Faculty clashes	One instructor cannot teach two classes simultaneously
Dual roles	Some faculty members also attend classes as students
Limited rooms	Several compatible classes may still compete for rooms
Limited slots	We cannot simply invent another color
Schedule quality	A valid schedule may still have poor gaps or workload spread

The Meaning of a Conflict Edge Expands



Shared students

Same faculty

Faculty is also a student

One graph edge can encode several different sources of conflict.

A Valid Coloring Can Still Be an Invalid Timetable

Suppose four mutually compatible classes are assigned the same time slot T_2.

None of the four classes conflict — so the coloring is valid.

But only three suitable rooms are available at T_2.

Result: the timetable cannot actually be executed.

Conflict-free coloring ≠ Feasible university schedule

Hard Constraints vs Soft Constraints

Hard constraints — MUST hold	Soft constraints — optimize if possible
No student clash	Good spread of teaching load
No faculty clash	Avoid very long gaps
Room available and suitable	Avoid overloaded days
Time slot permitted	Respect preferred slots where possible
Capacity / lab requirements	Reasonable free time for faculty

Question for the Class

Is university timetabling simply a Graph Coloring Problem?

Graph coloring gives the core conflict model

Resource + capacity + availability constraints

Optimization of schedule quality

Real University Timetabling

Algorithmic Starting Point: Greedy Coloring

Process one class at a time.

Check the time slots already used by its conflicting classes.

Assign the first available valid slot.

Continue until all classes are scheduled — or no valid slot remains.

color(v) = smallest available color not used by N(v)

Problem: the result depends strongly on the order in which classes are processed.

Improvement 1: Welsh–Powell

Schedule high-conflict classes first.

Compute the degree of each class in the conflict graph.

Sort classes by decreasing degree.

Color the classes in that order.

Interpretation: a class conflicting with 15 others deserves attention before one conflicting with only 2.

Degree ≈ how many other classes this class conflicts with

Improvement 2: DSATUR — Degree of Saturation

Which unscheduled class is becoming the hardest to schedule?

Saturation degree = number of different colors already used by a vertex's colored neighbors.

Choose the uncolored vertex with the highest saturation degree.

Break ties using ordinary degree.

Assign the lowest available valid color and update the saturation values.

Why DSATUR Is Relevant to Timetabling

Course	Total conflicts	Already blocked slots
CS101	8	2
CS301	6	4
SE401	10	3

Degree-first may choose SE401.

DSATUR chooses CS301: 4 different slots are already blocked.

But DSATUR Still Solves Only the Coloring Layer

DSATUR can produce a good conflict-free assignment of time slots.

It does not automatically guarantee enough rooms at each slot.

It does not automatically enforce room type/capacity, faculty availability, or preferred scheduling patterns.

It does not by itself optimize faculty gaps or weekly workload spread.

Coloring is the core — real timetabling is a constrained optimization problem.

Beyond Coloring: Real Timetabling Approaches



Constraint Programming / Constraint Satisfaction

Integer Linear Programming (ILP / MILP)

Heuristics and metaheuristics: tabu search, simulated annealing, genetic algorithms, etc.

Now: How Is It Actually Done Here?

We know the graph-theoretic model.

We know why the real problem is harder.

How does the actual university application produce a clash-free and well-balanced schedule?

How are conflicts represented?

Which classes are scheduled first?

How are rooms and shared faculty handled?

How are faculty gaps / free-time spread optimized?

What happens when no feasible slot is immediately available?

Discussion After the Demonstration

Which parts of the application correspond directly to graph coloring?

Does the system use a degree-based or 'most constrained first' strategy similar to DSATUR?

Which requirements behave as hard constraints?

Which requirements are optimization objectives?

Where does the practical solution go beyond graph theory?

Goal: connect a real production system back to the graph-theoretic ideas.